

Case Study:

Supabase Architecture Advisory

A shared audit platform

Project intake



Evidence and findings



Review



Deliverable

01

Design the audit workflow

A consultancy managed project audits through separate documents and manual workflows. I recommended a shared audit hub with reusable assessment rules, structured project records and clear client access. The advice gave the team a data model, service responsibilities and a practical first workflow to build.

Start with the work

The first step was to separate the consultancy's reusable method from each client's project data. The design questions were concrete:

- **Intake:** what information is needed before an audit starts?
- **Method:** which sections and assessment criteria repeat across projects?
- **Evidence:** how does a finding connect to the material that supports it?
- **Access:** who may contribute, review or approve work for this project?
- **History:** what should remain available after the deliverable is complete?

These answers define the records and permissions the platform needs. They also keep the first release focused on the audit process.

Choose services by responsibility

Supabase was the proposed starting point because the core needs were related records, user accounts, file storage and controlled server operations.

Need	Proposed service	Design rule
Clients, projects, audits and findings	PostgreSQL	Use relationships and stable record identities
Consultant and client accounts	Supabase Auth	Connect identity to explicit project membership
Documents and evidence files	Supabase Storage	Keep files private and link them to project records
Integrations and report requests	Edge Functions or backend handlers	Check permissions before privileged work

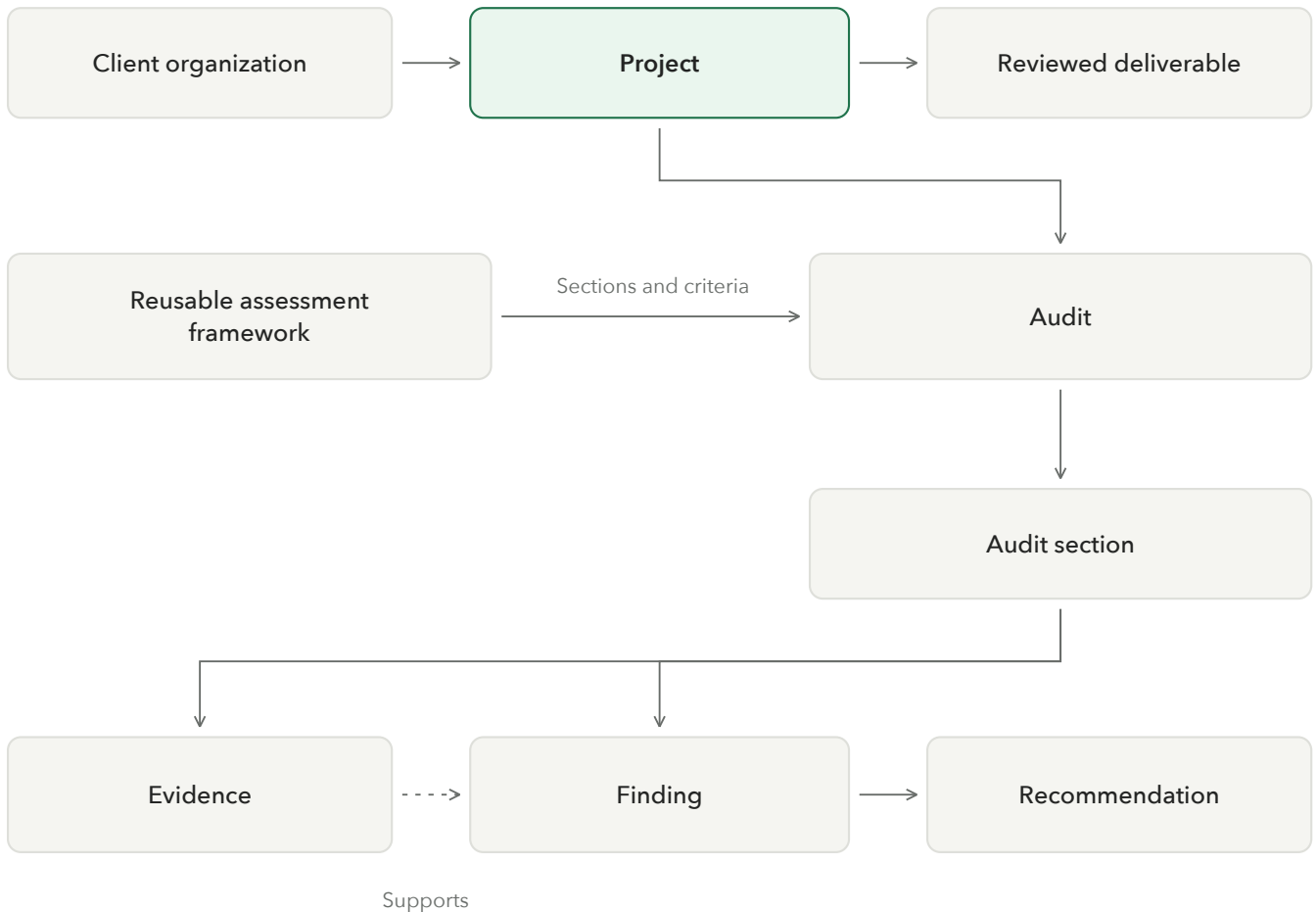
Platform choice also needs an operating plan: who manages access, monitors failures, handles recovery and reviews costs? For a team already using AWS, evaluate those same responsibilities against its existing services and skills. Add infrastructure when a workload needs it and someone can own its operation.

02

Proposed data model

Project intake > Evidence > Findings > Review > Deliverable

The proposed model gives each part of that flow a clear place:



Reuse the assessment structure while keeping project evidence and findings separate.

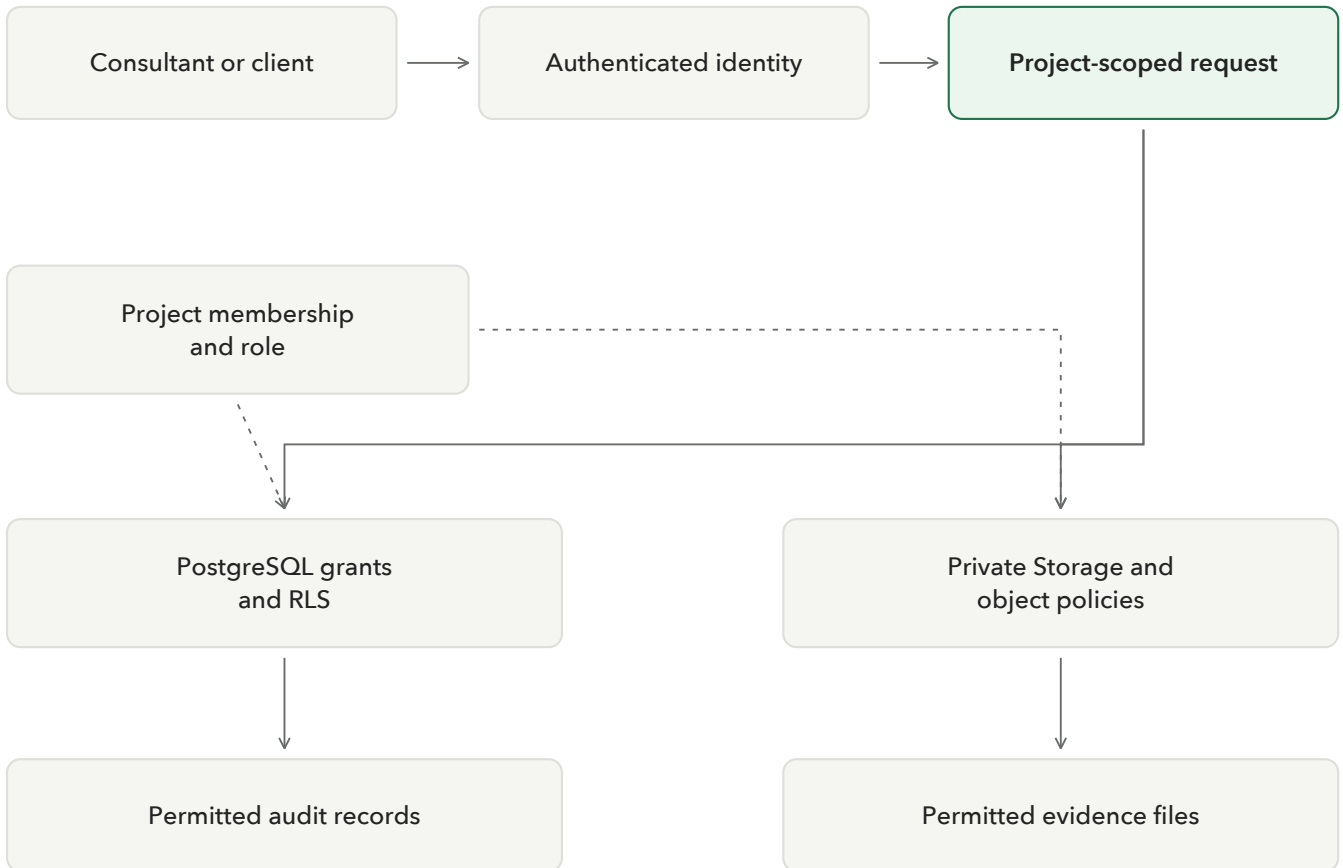
The implementation rules follow from that model:

- Give each project an organization and explicit member assignments.
- Link evidence, findings and recommendations to their audit context.
- Keep reusable criteria separate from project-specific answers.
- Preserve the criteria used for a completed audit when the method changes.
- Treat portfolio publication as a separate approval decision. Completing a client engagement does not make its evidence public.

This structure supports consistent audits and useful project history without requiring the team to recreate pages for every engagement.

Access rules and RLS

Sign-in establishes identity. Project membership and role determine what that person may do. The recommended access model makes those checks part of data access, including requests made outside the frontend.



The proposed database and file policies use the same membership model. Each resource still needs its own access rules.

The Supabase rules are:

- Enable RLS on every table in an exposed schema. Grant only the table operations the application needs.
- Write policies for reading, creating, updating and deleting records. Check both project membership and the requested action.
- For updates, check the existing row and the proposed values. Prevent an edit from moving a record into another client's project.
- Keep role assignments in controlled membership data. Users must not be able to grant themselves access through editable profile fields.
- Keep privileged keys on the backend. A handler using credentials that bypass RLS must perform its own actor and project checks.

The practical test is simple: a member can access the assigned project, but changing a project ID must not expose another client's records or files.

Storage and the first delivery

Private evidence files

Keep the file in object storage and its project, description and object reference in PostgreSQL. A database policy does not automatically protect the corresponding file.

- Use private buckets for client evidence.
- Define Storage policies for uploads, downloads, replacement and deletion.
- Issue short-lived download links only after checking access.
- Set upload size and file-type limits for the evidence the workflow accepts.

This gives staff useful file access without turning client documents into public assets.

Reports and integrations

Use server-side handlers for external credentials, administrative actions and report generation requests. Pass the actor and project context into each operation and check them before reading data or producing an output.

Match report execution to its workload. A small export may fit a request; a longer job needs saved status and a retry strategy. An interrupted request should not leave staff guessing whether a deliverable exists.

Build one complete workflow first

The recommended first delivery was one project with assigned members, evidence, findings and a reviewable result. Validate that path before adding more audit types or portfolio reporting.

The acceptance checks should cover:

- the correct members can complete each stage;
- users from another client cannot read or change its records and files;
- findings retain their evidence links through review;
- the deliverable reflects the reviewed project data.

Summary

The consultation turned a broad platform idea into a data model, an access model and a first delivery plan. The proposed hub reuses the consultancy's method while keeping client work separate. That gives the team a clear basis for implementation and a way to expand the product one verified workflow at a time.